# Projectile Motion Simulator

Physics Logic, Algorithms, and Project Info

ShOOmet

June 9, 2026

**Abstract**

This document explains the physics, numerical methods, validation logic, visualization logic, and overall architecture of the Projectile Motion Simulator project. The application compares projectile motion in three cases: no air resistance, linear drag, and quadratic drag solved with the fourth-order Runge-Kutta method. The current version also supports wind, light/dark themes, CSV export, plot export, GIF animation export, and vector visualization for wind and projectile velocity.

This is a desktop application written in Python with PySide6. Its purpose is to compare different projectile motion models and show how air resistance, wind, and physical parameters affect flight.

The user can change values such as initial speed, launch angle, mass, radius, drag coefficients, air density, time step, gravity, wind speed, and wind angle. After running the simulation, the application shows trajectory, energy, speed, playback animation, and text results. The user can export CSV files, PNG plots, and a GIF animation.

In simple words, drag is a force that acts against the motion of the projectile relative to the air. If there is wind, the projectile does not move through still air anymore. Instead, the air itself has velocity, so the drag depends on the difference between projectile velocity and wind velocity.

## 1 Coordinate system and angle convention

The application uses a two-dimensional Cartesian coordinate system with an $X$ axis and a $Y$ axis, where the x-axis represents horizontal position and the y-axis represents vertical position. The ground is represented by $y = 0$, and gravity pulls down in the direction of $-\hat{y}$.

The projectile starts at $(0, 0)$. The launch speed is $v_0$, and the launch angle is $\alpha$ measured above the positive $x$ direction (Quadrant I). The initial velocity components are

$$v_{x0} = v_0 \cos(\alpha)$$

$$v_{y0} = v_0 \sin(\alpha)$$

The wind angle is interpreted in the same coordinate system counterclockwise, where $0^\circ$ means wind blowing to the right, $90^\circ$ means wind upward, and so forth. For wind speed $w$ and wind angle $\phi$, the wind velocity components are

$$w_x = w \cos(\phi)$$

$$w_y = w \sin(\phi)$$

## 2 Parameters and validation

The physical parameters are stored in a frozen dataclass called `Parameters`. This gives the project one central place for parameter definitions, derived values, and validation.

| Parameter | Physical meaning | Validation |
|---|---|---|
| $y_0$ | Initial y position | $y_0 \geq 0$ |
| $x_0$ | Initial x position | $x_0 \geq 0$ |
| $v_0$ | Initial speed | $v_0 \geq 0$ |
| $\alpha$ | Launch angle | $\alpha \in [0°; 90°]$ |
| $m$ | Mass | $m > 0$ |
| $R$ | Projectile radius | $R > 0$ |
| $C_d$ | Quadratic drag coefficient | $C_d \geq 0$ |
| $\rho$ | Air density | $\rho \geq 0$ |
| $b$ | Linear drag coefficient | $b > 0$ |
| $\Delta t$ | Time step | $\Delta t > 0$ |
| $t_{\max}$ | Maximum simulation time | $t_{\max} > 0$ |
| $g$ | Gravitational acceleration | $g > 0$ |
| $w$ | Wind speed | $w \geq 0$ |
| $\phi$ | Wind angle | $\phi \in [0°; 360°]$ |

GUI Parameters panel has also upper limits on those values, but ther are less important than the general `Parameters @dataclass`. The validation also rejects non-finite numbers such as `nan` and `inf`. The application also protects itself from extremely large simulations using

$$\frac{t_{max}}{\Delta t} \leq \texttt{MAX\_SIMULATION\_STEPS}.$$

In the current project this maximum is set to one million steps. This prevents the program from freezing or using excessive memory when the time step is too small or the maximum time is too large. Inside the `@dataclass` object, several values are calculated from the base parameters. Below are the formulas used and what they represent.

- Cross-sectional area of the projectile is: $A = \pi R^2$
- Linear drag constant: $k = \frac{b}{m}$
- Quadratic drag constant: $q = \frac{\rho C_d A}{2m}$

The quadratic drag constant comes from the usual quadratic drag form:

$$F_d = \frac{1}{2}\rho C_d A v^2,$$

divided by mass to get acceleration.

# 3 No-drag model

The no-drag model assumes that the only force acting on the projectile is gravity. Wind has no effect in this model because there is no air resistance. The acceleration is only in the $-\hat{y}$ direction and it is $a_y = -g$. To calculate velocity, I use these formulas:

$$v_x(t) = v_{x0}.$$

$$v_y(t) = v_{y0} - gt.$$

And for the position:

$$x(t) = x_0 + v_{x0}t,$$
$$y(t) = y_0 + v_{y0}t - \frac{1}{2}gt^2.$$

This is the analytic solution, so it does not need numerical integration. The application evaluates these formulas over a time array created with `numpy.arange`.

# 4 Linear drag model with wind

The linear drag model assumes that drag acceleration is proportional to relative velocity between the projectile and the air. Wind is treated as moving air, so the relative velocity is

$$\vec{u} = \vec{v} - \vec{w}, \tag{1}$$

where $\vec{v}$ is the projectile velocity and $\vec{w}$ is the wind velocity. The acceleration equation is

$$\vec{a} = -k(\vec{v} - \vec{w}) + \begin{bmatrix} 0 \\ -g \end{bmatrix}.$$

Its components are:

$$\frac{dv_x}{dt} = -k(v_x - w_x),$$
$$\frac{dv_y}{dt} = -g - k(v_y - w_y).$$

The project uses the analytic solution for this model. The velocity formulas are

$$v_x(t) = w_x + (v_{x0} - w_x)e^{-kt}$$
$$v_y(t) = w_y - \frac{g}{k} + \left(v_{y0} - w_y + \frac{g}{k}\right)e^{-kt}.$$

The position formulas are

$$x(t) = x_0 + w_x t + \frac{(v_{x0} - w_x)(1 - e^{-kt})}{k}$$
$$y(t) = y_0 + \left(w_y - \frac{g}{k}\right)t + \frac{\left(v_{y0} - w_y + \frac{g}{k}\right)(1 - e^{-kt})}{k}$$

This model usually produces a shorter range than the no-drag case because the projectile loses horizontal speed over time. Wind can either reduce or increase the range depending on its direction.

# 5 Quadratic drag model with wind

The quadratic drag model is more realistic at higher speeds. Drag depends on the square of relative speed. The relative velocity is the same as for the linear-drag model, which is marked as equation 1. The relative speed is:

$$|\vec{u}| = \sqrt{(v_x - w_x)^2 + (v_y - w_y)^2}.$$

The acceleration caused by quadratic drag is

$$\vec{a}_{drag} = -q|\vec{u}|\vec{u}.$$

In components, the equations used in the code are

$$a_x = -q|\vec{u}|(v_x - w_x)$$
$$a_y = -g - q|\vec{u}|(v_y - w_y).$$

Unlike the no-drag and linear-drag models, the quadratic-drag model does not have a simple closed-form solution for this project. Therefore it is solved numerically.

## 5.1 State vector and Runge-Kutta method

For this numerical solving, we need to implement some sort of integration method. I used the fourth-order Runge-Kutta method, which I will call RK4 later. Another solution that would be enough for this project would be Euler's integration method. It is the first-order version of the RK method. I built the project with it first, but then I decided to move to a more accurate method. The quadratic-drag solver uses a state vector

$$\vec{s} = \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix}$$

The derivative of the state is

$$\frac{d\vec{s}}{dt} = \begin{bmatrix} v_x \\ v_y \\ a_x \\ a_y \end{bmatrix}$$

The application advances this state using the RK4 method. For a derivative function $f(t, \vec{s})$, time step $\Delta t$, and state $\vec{s}_n$, RK4 computes

$$k_1 = f(t_n, \vec{s}_n)$$
$$k_2 = f\left(t_n + \frac{\Delta t}{2}, \vec{s}_n + \frac{\Delta t}{2}k_1\right)$$
$$k_3 = f\left(t_n + \frac{\Delta t}{2}, \vec{s}_n + \frac{\Delta t}{2}k_2\right)$$
$$k_4 = f(t_n + \Delta t, \vec{s}_n + \Delta t k_3)$$
$$\vec{s}_{n+1} = \vec{s}_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

In the project, the derivative does not explicitly depend on time, only on the current state and parameters, so the function only receives the state and physical constants.

### 5.2 Pseudocode for quadratic drag

```
state = [x0, y0, vx0, vy0]
time = 0

while y >= 0:
    state = rk4_step(state)
    time += dt
    store state and time

    if time >= t_max:
        raise error
```

The loop stops once the new $y$ value is below ground. The final point is then corrected so that the trajectory ends exactly at $y = 0$ using interpolation, explained below.

### 5.3 Ground-hit interpolation

The simulation works in discrete time steps. Because of this, the projectile usually does not land exactly on $y = 0$ at one of the sampled time values. Instead, the solver finds the first value below ground and interpolates between the last above-ground point and the first below-ground point. For any value $z$ that should be corrected at ground impact, the interpolation formula is

$$z_{hit} = z_1 + \frac{-y_1}{y_2 - y_1}(z_2 - z_1),$$

where point 1 is above the ground and point 2 is below the ground. The same formula is used for impact time, $x$, $v_x$, and $v_y$. This improves the final range and flight time because the result does not simply stop at the last sampled point.

## 6 Speed and energy calculations

After calculating position and velocity, the project calculates speed, kinetic, potential, and mechanical energies as follows:

$$v = \sqrt{v_x^2 + v_y^2}$$

$$E_k = \frac{1}{2}mv^2$$

$$E_p = mgy$$

$$E = E_k + E_p$$

In the no-drag model, mechanical energy should stay approximately constant except for small numerical and interpolation effects. In drag models, mechanical energy decreases because drag removes mechanical energy from the projectile and converts it into heat and air motion.

## 7 GUI, Settings, and Project structure

The project is divided into separate modules. The config module stores parameters and settings. The simulation module contains the physics calculations. The GUI module contains the PySide6 interface. The visualization module exports plots and GIF animations. The storage module handles CSV export. This separation makes the project easier to understand and modify. You can find examples of the GUI, Settings panel, and project architecture on my GitHub project repository.